Franklin Open 7 (2024) 100115

Contents lists available at ScienceDirect

Franklin Open

journal homepage: www.elsevier.com/locate/fraope

Support Vector Machine reconfigurable hardware implementation on FPGA

Mohammed H. Yacoub [a], Samar M. Ismail [b,*], Lobna A. Said [a], Ahmed H. Madian [d], Ahmed G. Radwan [c,e]

[a] Nanoelectronics Integrated Systems Center (NISC), Nile University, 12588, Giza, Egypt
[b] School of Engineering and Computer Science, University of Hertfordshire, Egypt
[c] Engineering Mathematics and Physics Department, Faculty of Engineering, Cairo University, 12613, Giza, Egypt
[d] Radiation Engineering Department, National Centre for Radiation Research and Technology, Egyptian Atomic Energy Authority, 11787, Cairo, Egypt
[e] School of Engineering and Applied Sciences, Nile University, 12588, Giza, Egypt

ARTICLE INFO

Keywords:
Artificial intelligence
Support Vector Machine
Classification
Floating point
FPGA

ABSTRACT

Support Vector Machine (SVM) is a robust Machine Learning (ML) algorithm used extensively in classification tasks. This work proposes a reconfigurable hardware implementation of the SVM classification algorithm for the linear and three kernel cases on FPGA. Efficient implementations of two generalization techniques, One-versus-All (OvA) and One-versus-One (OvO), to deal with multi-class problems are also realized on FPGA to overcome the binary nature of the SVM algorithm. The presented model is fully reconfigurable and can easily be adapted to any dataset with any number of classes or features. The results show that the proposed model excels in power efficiency, requires low area utilization, and reaches high performance up to 250.7 MHz. The two realized generalization methods, OvO and OvA, offer a trade-off between accuracy and hardware cost. OvA provides lower accuracy than OvO and is more affected by the data imbalance problem, which becomes more dominant as the number of classes increases; however, it is more resource-efficient than OvO. The proposed hardware design is realized and experimentally tested on FPGA for three different datasets, employing different linear and kernel SVM cases. The classification accuracy achieved by the reconfigurable hardware design matches the software simulation results.

1. Introduction

Artificial intelligence (AI) is the field of science that enables machines to mimic the problem-solving and decision-making capabilities of the human brain [1]. The use of AI has invaded today's life in various fields such as plant technology [2], medical services and applications [3], security [4], robotics [5], spam detection [6] and much more. Machine learning (ML) is an AI sector that depends on data and algorithms to imitate the brain's learning process. Various ML algorithms exist with different levels of complexity and suitability for certain applications than others including Decision Tree (DT) [7], Random Forest [8], Naive Bayes [9], K- Nearest Neighbors (KNN) [10], Convolutional Neural Networks (CNNs) [11], and Support Vector Machine (SVM) [12].

SVM is a robust machine learning algorithm, especially when dealing with complex small or medium-size datasets [13]. The SVM algorithm is based on fitting the optimal border between the different classes of the dataset that keeps the largest distance between the different classes' border instances. SVM can be used for linear and non-linear classification. SVM is also utilized in regression by shifting the task from fitting the line with the optimal margin with the lowest possible sample lying in the margin area to fitting the largest number of samples between the margin [14]. SVM can also detect outlier samples in the dataset [13] and is also suitable for clustering [15]. This work discusses the usage of the SVM algorithm in classification tasks only by either applying a linear or a kernel method. This work also discusses the generalization of the SVM algorithm for multi-class problems using OvA and OvO techniques and proposes a reconfigurable realization of the algorithm on FPGAs.

In many recent publications, SVM was employed in various applications for classification and regression tasks [16–18]. In [19], the use of SVM was presented to enhance the diagnosis of tuberculosis infection. The authors in [20] proposed an ensemble learning model based on SVM for breast cancer diagnosis. They utilized C-SVM and υ-SVM models with different kernel functions to increase the diversity of the base model set. A classification of an SVM-based approach to detect kidney stones from the ultrasound image dataset was illustrated in [21].

* Corresponding author.
E-mail addresses: m.hassan2127@nu.edu.eg (M.H. Yacoub), s.m.ismail@ieee.org (S.M. Ismail), l.a.said@ieee.org (L.A. Said), amadian@nu.edu.eg (A.H. Madian), agradwan@ieee.org (A.G. Radwan).

https://doi.org/10.1016/j.fraope.2024.100115
Received 21 September 2023; Received in revised form 5 May 2024; Accepted 27 May 2024
Available online 29 May 2024

The authors in [22] developed a Machine Learning-Assisted Network Inference (MALANI) algorithm utilizing SVM to uncover Class II cancer genes in coordinating functionality in cancer networks that differ from Class I genes as they are neither mutated nor differentially expressed. These characteristics allow them to escape detection by analytical tools. A three-stage method for recognizing Arabic letters, starting with character extraction, then feature extraction using the Histograms of Oriented Gradient (HOG) method, ending with classification based on an SVM model, was presented in [23]. The authors in [24] proposed an ensemble learning based on SVM to detect the human face in digital videos. They reached a skin/non-skin classification accuracy of about 98% and a face/non-face classification rate above 94%. A weed detection algorithm for sesame crops based on Region-Based CNN utilizing SVM to improve the classification accuracy was proposed in [25].

Hardware realization is employed in many applications in various fields [26–29] to improve the performance at strict power consumption environments and benefit from the hardware's parallelism capability. Hardware solutions, in general, provide high speed, high function parallelism capabilities, small area, and efficient power consumption. Popular hardware solutions are FPGA and Application Specific Integrated Circuit (ASIC). FPGAs are popular in research due to their short development cycle and programmability compared to ASICs. Using FPGAs is also a low-cost solution [30]. Furthermore, FPGAs have a much faster time to market [31].

This work provides, with a generic methodology, a reconfigurable hardware realization of SVM for variable dataset dimensions, number of support vectors and features, with linear (no kernel), Radial Basis Function (RBF), polynomial, and sigmoid kernels. Each kernel design is implemented on hardware employing both fixed point and standard IEEE754 single precision floating point arithmetic units. Also, two generalization techniques One versus All (OvA) and One versus One (OvO) are implemented to compensate for the binary nature of the SVM algorithms. All the described architectures are realized and tested on two different FPGAs with different specifications and on three datasets with various dimensions to ensure the architectures' reconfigurability and compatibility with different boards. The rest of the paper is organized as follows: Section 2 presents an overview of the linear and kernel binary SVM, as well as the multi-class classification methods, OVA and OVO, used to generalize the binary SVM model. Section 3 discusses the software simulation procedure and performance metric evaluation of SVM for the different kernels and multi-class techniques. Section 4 proposes the hardware architecture, implementation, performance analysis, and experimental testing of the developed SVM reconfigurable architecture. Finally, Section 5 concludes this work.

2. Overview of SVM algorithm

SVM classification is based on fitting the optimal hyperplane that maximizes the margin between each class's border samples (Support Vectors) as shown in Fig. 1. SVM is considered a binary algorithm applicable to datasets with only two classes. However, the SVM concept can be extended to work with multi-class problems using the Multi-class Classification Methods, such as One-versus-All (OvA) and One-versus-One (OvO) discussed later in this section.

2.1. Linear SVM

Linear support vector machine (SVM) is a binary classification technique that predicts a sample X for being in a certain class or not according to the predicted value given by (1).

$$Prediction = W^T X + b, \tag{1a}$$

$$Y_{predict} = \begin{cases} 1 & \text{if Prediction} > 0, \\ 0 & \text{otherwise,} \end{cases} \tag{1b}$$

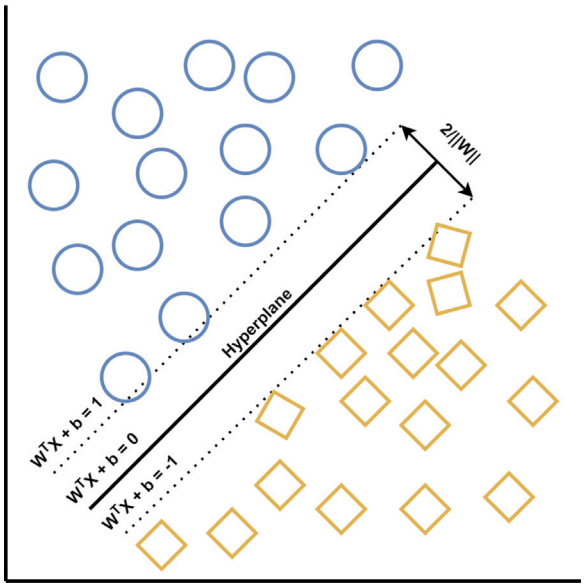


Fig. 1. Support Vector Machine optimal hyperplane between a binary dataset's samples.

where W is the weights vector, X is the sample's features vector, and b is the bias. By simplifying (1), the decision boundary can be written as in (2) where X_i is the training sample i, Y_i training label i, M is the number of training samples. The boundary width is given by $2/\|W\|$ which is aimed to be maximized.

$$Y_i(W^T X_i + b) \geq 1, \text{where } 1 \leq i \leq M. \tag{2}$$

To obtain the optimal hyperplane, the Lagrange multipliers are used to maximize the decision margins by minimizing W under the boundary constraint given by (2). The Lagrange dual form equation for SVM training is given by (3) and is subjected to the two constraints given by (3b) and (3c):

$$Maximize\ L,\ L = \sum_{i=1}^{M} \alpha_i - \sum_{i,j=1}^{M} \alpha_i \alpha_j Y_i Y_j X_i^T X_j, \tag{3a}$$

$$\sum_{i=1}^{M} \alpha_i Y_i = 0, \tag{3b}$$

$$0 \leq \alpha_i \leq C. \tag{3c}$$

where M is the number of samples, C is an optimization parameter defining the margin's strictness (i.e. the permittivity of a sample to lie in the margin), and α's are the Lagrange multipliers. The Lagrange multipliers are calculated using Quadratic Programming (QP), especially Sequential Minimal Optimization (SMO). SMO breaks a QP problem into small QP problems and then utilizes these problems to increase the speed of the evaluation as these small problems are solved quickly and analytically, significantly improving its scaling and computation time [32].

The calculated support vectors can be used directly for prediction. In linear SVM, the weights vector (W) can be calculated from the support vectors first and be used for prediction instead to improve the performance and reduce the computation complexity of the testing phase. The weights vector is given by:

$$W = \sum_{i=1}^{S} \alpha_i Y_i X_i, \tag{4}$$

where S is the number of support vectors, and X_i are the support vectors defined as the data samples with non-zero Lagrange multipliers.

Finally, the bias (b) is straightforwardly calculated based on the acquired weights and the nearest two opposite samples to the hyperplane and is given by:

$$b = -\frac{min_{i:y_i=1}(W^T X_i) + max_{i:y_i=-1}(W^T X_i)}{2}. \quad (5)$$

2.2. Non-linear SVM

In many cases, datasets' classes are not linearly separable. Adding non-linear features can improve the performance in such cases. However, this requires mapping the dataset to a higher dimension with added non-linear features, which highly increases the required calculation, increasing the resources and the classification time. The kernel trick is used in SVM to avoid the dimensionality problem. A kernel is a similarity measure [33] that eliminates the need to perform the mapping operation as it provides the result of the dot product in the mapped dimension. The kernel's general equation is given by:

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j), \quad (6)$$

where $\phi(x_i), \phi(x_j)$ represents two samples x_i, x_j mapped to another dimension. A function is considered a kernel if it satisfies Mercer's condition [34].

A variety of kernels exist. Some of them are for a specific task, such as the string kernel, which operates specifically on strings, and the rest of them are for generic tasks like the Gaussian kernel. This work discusses only RBF, Polynomial, and Sigmoid kernels besides the linear non-kernel realization. In kernel SVM, calculating the weights vector is not applicable as the mapping is not performed as it is exceedingly resource-consuming, which is the reason behind using kernels. Instead of that, the prediction is performed using the support vectors directly. To make the design methodology more generic, the support vectors are also used to classify the test samples in the linear SVM as well. The prediction in a kernel SVM is given by:

$$Prediction = \sum_{i=1}^{S} Y_i \alpha_i K(D, X_i) + b, \quad (7)$$

where S is the number of support vectors, D is the data sample, X_i is the ith support vector, Y_i is the target of the ith support vector, α_i is the dual coefficient of the ith support vector, and b is the intercept.

2.2.1. RBF kernel

RBF kernel is a simpler form of the Gaussian kernel given by:

$$K(x_i, x_j) = \exp^{(\|X_i, X_j\|^2/2\sigma^2)}. \quad (8)$$

RBF kernel is a popular kernel for classifying non-linearly separable datasets. It is defined as the dot product of infinite dimensions mapped vectors. The RBF kernel is given by:

$$K(x_i, x_j) = \exp^{(-\gamma \cdot \|X_i, X_j\|^2)}, \gamma > 0, \quad (9)$$

where γ is an optimization parameter that defines the spread of the kernel.

2.2.2. Polynomial kernel

The polynomial kernel defines the output of the dot product of two vectors mapped to a higher dimension by adding polynomial features. The number of features grows exponentially by increasing the degree d ($x \mapsto x^d$). In many cases, adding polynomial features can improve the algorithm's performance; however, it can lead to over-fitting. Only the 3rd degree polynomial is implemented in this work. The polynomial kernel is given by:

$$K(x_i, x_j) = (\gamma \cdot X_i^T X_j + r)^d, \gamma > 0, \quad (10)$$

where γ and r are optimization parameters.

2.2.3. Sigmoid kernel

The sigmoid kernel is another popular kernel that is based on the tanh function, not on the logistic sigmoid function as the name implies. The sigmoid kernel is given by:

$$K(x_i, x_j) = tanh(\gamma \cdot X_i^T X_j + r), \gamma > 0, \quad (11)$$

where γ and r are optimization parameters.

2.3. Multi-class classification techniques

In most cases, a classification problem includes more than two classes. As stated before, SVM is a binary classifier that only predicts the existence or non-existence of a data sample in a certain class. Multi-class classification methods generalize the algorithm to any number of classes. This section discusses One-versus-All (OvA) and One-versus-One (OvO) techniques.

2.3.1. One-versus-all

OvA classifies between each specific class and the rest of the training dataset. The data is first divided into two sets; class and non-class. Each sample is then labeled "1" if it belongs to this class, and the rest of the training data is labeled "-1". This process is performed for each class against the rest of the data, generating a total of C classifiers where C is the number of classes in the dataset. Each classifier is trained, and then each test sample is passed to each classifier. The classifier with the highest prediction result assigns the sample to its associated class.

There exist two drawbacks to this method; the first is the data imbalance problem as each classifier, for an equally distrusted set of c classes and m samples, trains a set divided between two classes the first of (m/c) samples and the second of $((c-1)m/c)$ samples. So, this approach does not apply to datasets containing a huge number of classes where the imbalance problem becomes more severe. The second drawback is the unclassified regions. While the classifier tries to fit the widest margin between the samples, it leaves behind unclassified regions represented in Fig. 2. These unclassified regions are visible in the prediction of each classifier. For some samples, the prediction of all the classifiers is less than zero, implying that the sample lies in the unclassified region. For the OvA technique, this unclassified region is larger than OvO as the algorithm is classifying a class versus the rest of the data, not against another class.

The main advantage of the OvA technique is that it requires only C classifiers, for a dataset of C classes, hence consuming fewer resources than the OvO technique. This also implies a shorter time for both training and prediction. OvA is preferred for balanced datasets with a relatively small number of classes.

2.3.2. One-versus-one

OvO trains a unique classifier for every two classes i and j where $i \neq j$. Classifying one class against another solves the data imbalance problem for balanced datasets and mitigates the unclassified region problem as in Fig. 2, which shows that the unclassified region shrinks significantly when OvO is used. If the data distribution is non-uniform from the beginning, the imbalance problem will still exist. OvO is more resource-consuming than OvA as training a dataset containing C classes requires training $C(C-1)/2$ classifiers. OvO can handle imbalanced datasets better than OvA but becomes computationally expensive for datasets with a large number of classes.

3. Software simulation

In order to evaluate the performance of the proposed reconfigurable hardware, three datasets were considered as a reference for testing the classification results. These datasets are selected so that the first, the Heart Disease dataset, is a binary dataset containing only two classes. The second, The Iris dataset, is a multi-class dataset slightly affected by data imbalance. The third dataset, the Mobile Prices, is

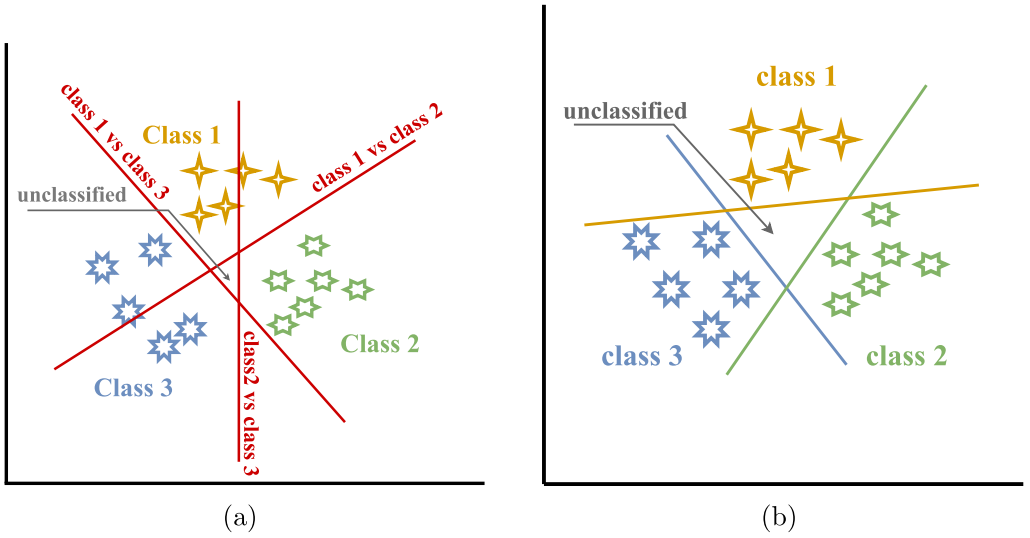


Fig. 2. Multi-Class Techniques (a) OvO, and (b) OvA.

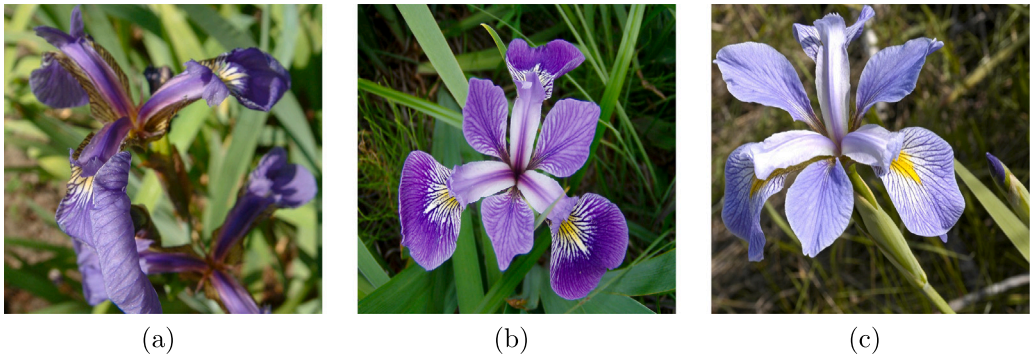


Fig. 3. Iris dataset classes (a) Iris Setosa, (b) Iris Versicolour, and (c) Iris Virginica.

significantly affected by the non-uniform data distribution nature of the OvA classifiers. A procedure based on 5-fold cross-validation is employed to optimize the hyper-parameters used in the training process for each dataset, kernel, and multi-class method case.

3.1. Datasets

3.1.1. Heart disease dataset

The Heart Disease dataset consists of 303 samples, each having 13 features. The binary classes represent the presence of heart disease in the patient when "1" and the absence when "0". The features of each sample are age, sex, chest pain type, resting blood pressure, serum cholesterol, fasting blood sugar, resting electrocardiographic results, maximum heart rate achieved, exercise-induced angina, old peak, the slope of the peak exercise ST segment, number of major vessels, and thal [35]. Since this dataset is binary, a multi-class technique will not be applicable, as only a single classifier will be required.

3.1.2. Iris dataset

The Iris dataset is a balanced set consisting of three classes, each representing a type of Iris flower. The dataset comprises 150 samples, each having four features. These features are sepal length in cm, sepal width in cm, petal length in cm, and petal width in cm. The dataset classes are Iris Setosa, Iris Versicolour, and Iris Virginica [36]. A sample image of each class is shown in Fig. 3. Fig. 4 visualizes two features of the dataset and shows the classification of the dataset based on these two features only applying linear, RBF, and polynomial classifiers, respectively. The visualization shows that only two classes on the set, the red and blue dots, are linearly separable, unlike the third one. Visualizing the dataset even partially can give information on which kernel is more suitable to be applied to the dataset in hand. The Iris dataset is small and balanced, so both OvA and OvO can perform well in its classification.

3.1.3. Mobile prices dataset

The dataset categorizes 2000 mobile phones into four price ranges based on their specifications. [37]. Each phone sample in the dataset is represented by 20 features, including battery power, Bluetooth, clock speed, dual SIM support, front camera megapixels, 4G support, internal memory size, mobile depth in cm, phone weight, number of cores, primary camera megapixels, resolution height, resolution width, RAM size, screen height, screen width, talk time with a single battery charge, 3G support, touch screen availability, and WiFi support. Each sample is labeled into one of four classes; "0", "1", "2", and "3", where "3" represents the highest price range. When using the OvA technique with this dataset, the imbalance problem becomes more noticeable, and it affects the accuracy compared to OvO, given the relatively large number of samples in each of the four classes.

3.2. Results

The classification results for each dataset applying kernel and non-kernel SVM with both multi-class methods are evaluated using accuracy, precision, recall, and F1-score metrics, which are given by:

$$Accuracy = \frac{TN + TP}{TN + TP + FN + FP}, \tag{12a}$$

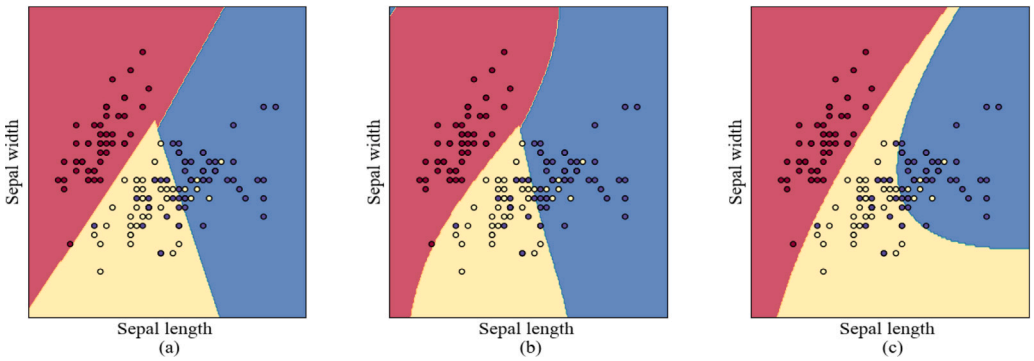


Fig. 4. Iris dataset classified and visualized based on 2 of 4 features only (Sepal length and Sepal width) with (a) Linear, (b) RBF, and (c) Polynomial kernel SVM classifiers. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 1
Classification results.

Dataset	Multi-Class Method	Linear				RBF			
		Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score	Accuracy
IRIS	**OvO**	1.00	1.00	1.00	100%	1.00	1.00	1.00	100%
	OvA	0.97	0.97	0.97	96.67%	1.00	1.00	1.00	100%
MOBILE	**OvO**	0.95	0.95	0.95	94.75%	0.93	0.93	0.93	93%
	OvA	0.76	0.77	0.76	77%	0.81	0.81	0.81	81.25%
HEART		0.82	0.81	0.81	82%	0.86	0.84	0.85	85%
Dataset	Multi-Class Method	Polynomial 3rd Degree				Sigmoid			
		Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score	Accuracy
IRIS	**OvO**	0.92	0.90	0.89	90%	0.87	0.83	0.84	83.33%
	OvA	1.00	1.00	1.00	100%	0.91	0.90	0.91	90%
MOBILE	**OvO**	0.82	0.82	0.81	81.25%	0.92	0.92	0.92	92.25%
	OvA	0.67	0.68	0.66	68.25%	0.67	0.69	0.67	69.5%
HEART		0.82	0.82	0.82	82%	0.85	0.84	0.83	83.6%

$$Precision = \frac{TP}{TP + FP}, \tag{12b}$$

$$Recall = \frac{TP}{TP + FN}, \tag{12c}$$

$$F-1\ Score = \frac{Precision * Recall}{Precision + Recall}. \tag{12d}$$

where TP is the number of true positive samples, FP is the number of false positive samples, TN is the number of true negative samples, and FN is the number of false negative samples. The evaluated results are shown in Table 1, and confusion matrices for each classification method are shown in Table 2. The results show that each dataset's properties affect its classification rate. The Iris dataset is small and balanced, so both multi-class techniques achieved their optimal margin. Unlike the Iris dataset case, the Mobile dataset is much larger, so even though it has a balanced distribution (500 samples for each class), this balanced state is not kept during training with the OvA. Using OvA, each classifier classifies an average of 400 samples against 1200 samples, affecting the classification performance. This problem is not encountered when using the OvO algorithm, as it classifies one class against another. In most cases, using linear SVM is enough to get appropriate results for the datasets tested in this work. The RBF kernel produced the best classification results for kernel SVM. Based on these results, linear SVM with the OvA algorithm should be tried first, as it is the most efficient technique. Based on the required performance, dataset properties, and available resources, RBF Kernel with OvO algorithm could offer a better option, especially for linearly inseparable and imbalanced datasets.

4. Hardware implementation

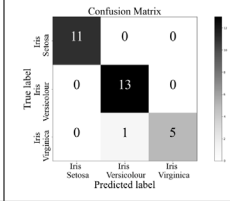
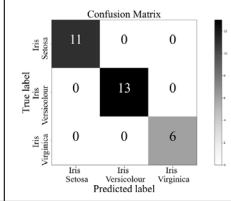
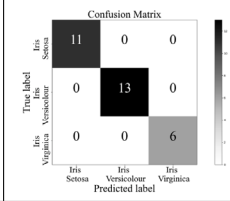
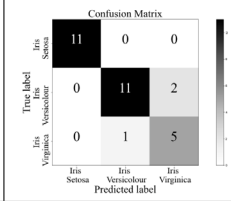
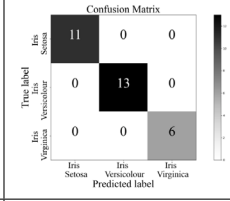
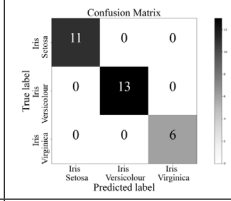
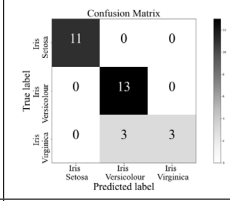
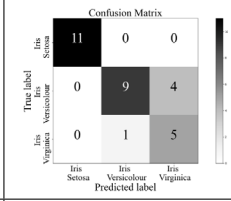
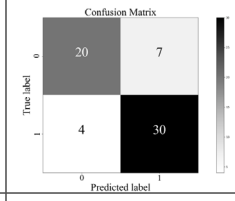
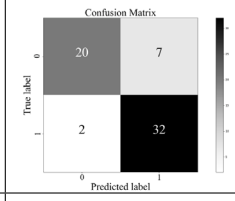
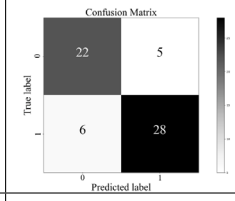
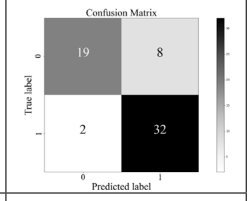
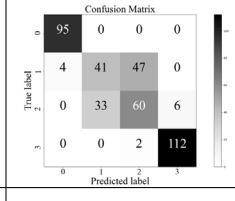
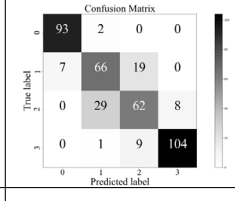
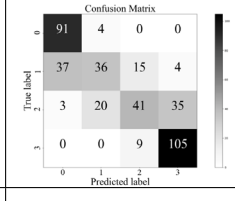
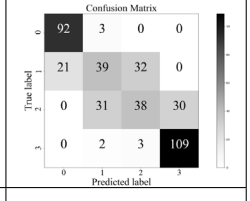
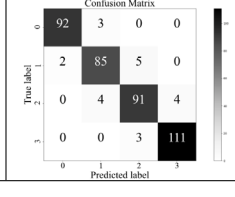
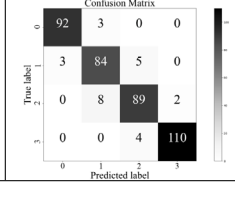
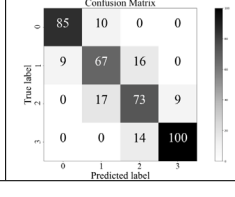
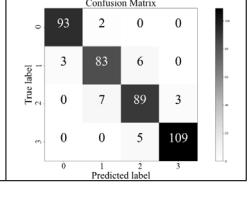
Reconfigurable hardware implementations of linear and kernel SVM with two multi-class generalization methods, OvA and OvO, are designed and tested on two of Xilinx's FPGAs, Kintex-7 and Artix-7. The hardware designs employ IEEE 754 [38] single-precision (SP) floating-point (FP) as well as variable length fixed-point arithmetic. FP arithmetic, represented in Fig. 5, has the advantage of developing a universal design while minimizing the number of bits needed to represent large numbers. FP representation also offers higher precision and accuracy than the fixed-point (FX) representation of the same length [39]. Single precision is used instead of double precision as it offers high accuracy with less hardware cost [40]. Fig. 6 depicts the FP adder architecture based on Carry Look ahead adder (CLA) to decrease latency [41] and Fig. 7 presents the FP multiplier hardware architecture. Both architectures are combinational, requiring only a single cycle to execute.

The hardware design benefits from the parallelism and pipeline capabilities of the FPGA fabric, unlike the software sequential case, hugely reducing the complexity of the design. The resources of the FPGA are utilized to employ a unique pipeline path for each classifier, where each path represents a complete pipeline, which significantly increases the throughput of the design. Furthermore, simple polynomial approximations were employed to represent the kernel SVM's nonlinear functions to reduce the algorithm's computational complexity with minimal accuracy reduction. The design also includes a variable fixed point arithmetic unit soft macro block. This block can be used instead of the high-accuracy FP unit to further reduce the design complexity based on the required application and the targeted accuracy. The rest of the section is organized as follows: first, the hardware design of the SVM is illustrated, then the hardware utilization is presented and compared with previous implementations found in the literature. Finally, the hardware experimental results are provided.

4.1. SVM hardware design

Several hardware components are designed for the implementation of the SVM architecture. Both multi-class techniques, OvA and OvO,

Table 2
Confusion Matrices of each approach.

	Linear	RBF	Polynomial	Sigmoid
Iris (OvA)				
Iris (OvO)				
Heart				
Mobile (OvA)				
Mobile (OvO)				

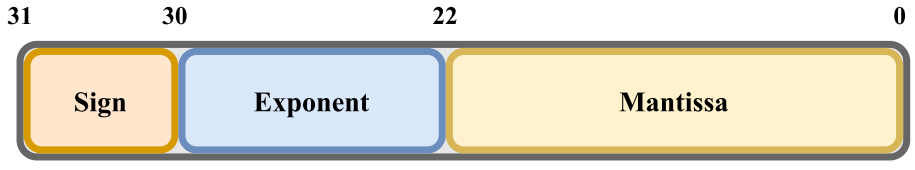


Fig. 5. Single precision floating point representation.

are implemented on hardware. The main difference between their implementations is that The OvO method utilizes a majority picker component that can detect the most frequently selected class by the classifiers. On the other hand, the OvA requires a maximum number detector to detect the class that its classifier's prediction result is the maximum and assigns the sample to that class. Fig. 8 shows the block diagram of OvA and OvO, where C is the number of classes, and N is the number of classifiers, equals C for OvA and $\frac{C(C-1)}{2}$ for OvO.

4.1.1. Linear SVM design

The linear SVM hardware architecture starts with calculating the dot product between the data sample and each of the support vectors of the dataset as shown in Fig. 9, where SV_0^n, for example, is the nth feature of the 0th support vector. The result of each dot product is multiplied by the dual coefficient $DC_i = Y_i\alpha_i$ associated with this support vector and accumulated on the previous summation. Finally, the intercept is added to the final accumulated result in the register to form the prediction of this classifier. The complete architecture of the Linear SVM for multi-class problems is depicted in Fig. 10.

4.1.2. RBF kernel SVM design

Two operations are carried out to calculate the RBF kernel. First, the squared norm of the difference between each support vector and the data sample is computed, and the result is then multiplied by the hyperparameter γ. The second operation is taking the exponent of the outcome of the first operation. Fig. 11 shows the hardware block diagram of the first operation ($\gamma\|X_i - X_j\|^2$). The exponential function is approximated for hardware implementation based on a 9-term 8th

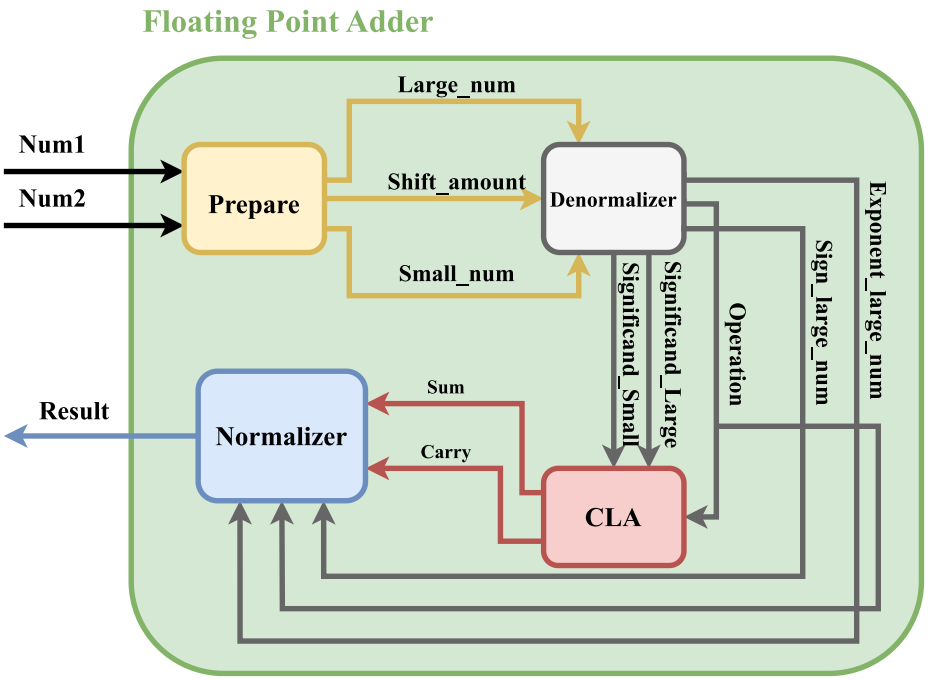


Fig. 6. Floating point adder block diagram.

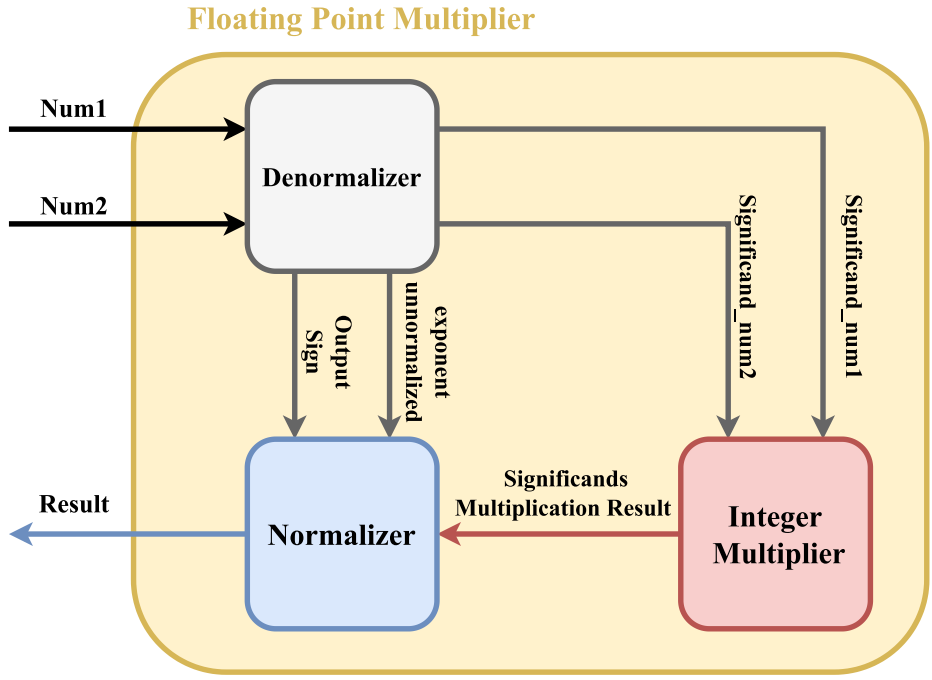
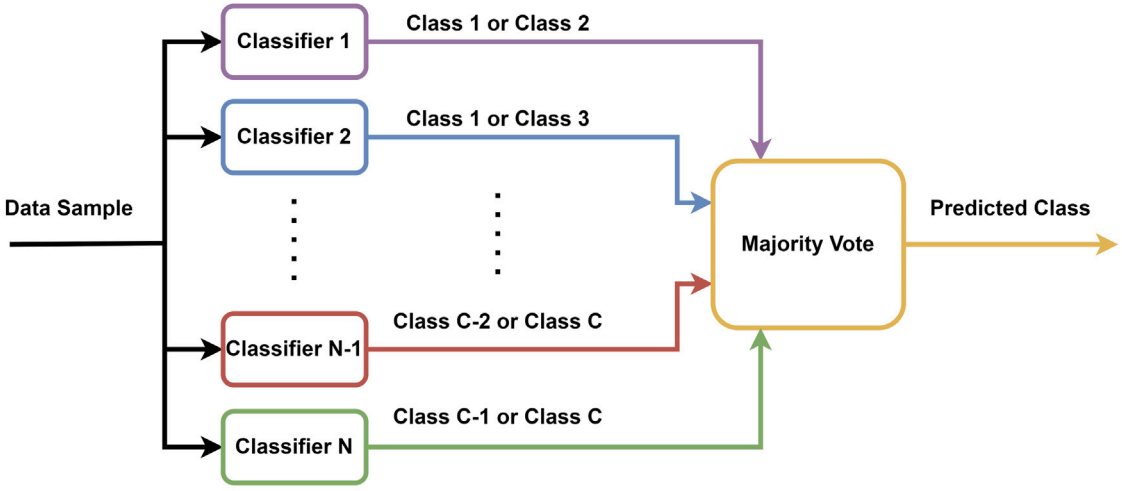


Fig. 7. Floating point multiplier block diagram.

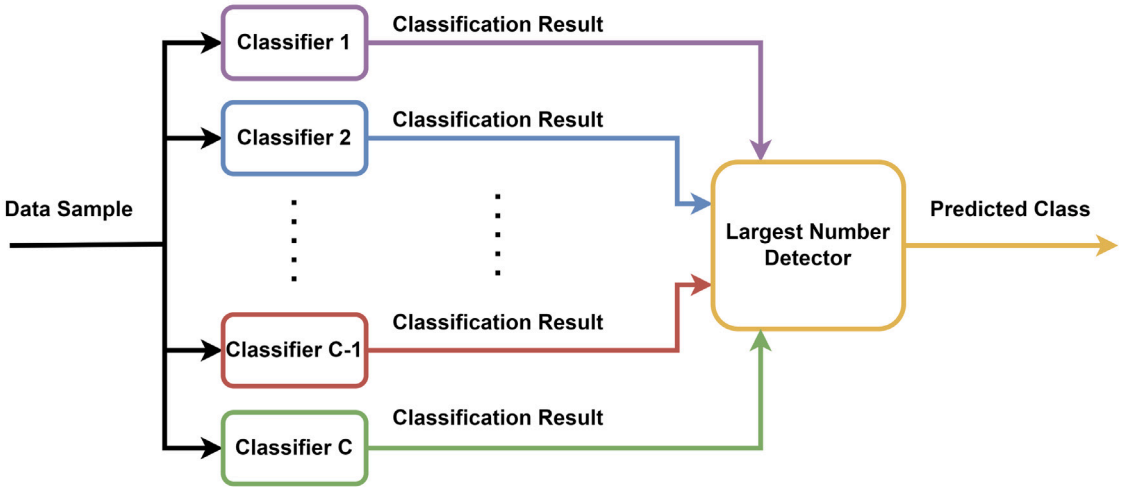
degree polynomial approximation given by:

$$e^x = \begin{cases} c_8x^8 + c_7x^7 + c_6x^6 \\ + c_5x^5 + c_4x^4 + c_3x^3 \\ +c_2x^2 + c_1x + c_0 & , -9.284 < x \leq 0 \\ e^{-9.284} & , x < -9.284. \end{cases} \tag{13}$$

The approximation coefficients are listed in Table 3. The approximation fits the range from −10 to 0 well. Inputs less than −9.284 are approximated to the result for −9.284, where the result will not significantly affect the classification accuracy as the intercept value will dominate in the final computation. The hardware block diagram of the exponential function is shown in Fig. 12. A comparison between the proposed approximation and Taylor's approximation of the 20th degree in the same region is presented in Table 4, showing the superiority of the proposed approximation in both accuracy and resources. Figs. 13(a) and 13(b) show the proposed approximation and Taylor's approximation, respectively, plotted overlaid on the exact results calculated using Python's math library. The RBF SVM calculation is as follows: the kernel is calculated for each support vector and multiplied by the respective dual coefficient. The results for all the support vectors are accumulated. Finally, the intercept term is added to the accumulated



(a)



(b)

Fig. 8. Multi-Class Techniques Block Diagrams (a) OvO, and (b) OvA.

Table 3
Exponent approximation coefficients.

Coeff	Value	Coeff	Value
c_0	0.9991004426	c_5	0.003851537754
c_1	0.9913063420	c_6	0.00032268521126
c_2	0.4790764651	c_7	$1.537970590*10^{-5}$
c_3	0.14466839051	c_8	$3.1655628814*10^{-7}$
c_4	0.029028528878		

Table 4
Exponent approximations comparison.

Metric	Propsed	20th Taylor
MSE	$4.6719*10^{-9}$	$1.8699*10^{29}$
Multipliers	15	39
Adders	8	20

result, forming the prediction of a certain classifier. The Complete RBF SVM for multi-class problems is shown in Fig. 14.

4.1.3. Polynomial kernel SVM design

The polynomial kernel shares the dot product part with the linear kernel. The dot product output is then multiplied by the hyperparameter γ, the output is added to the hyperparameter r, then raised to the power d according to the polynomial degree used, and the output is the kernel's result. Besides the kernel calculation, the polynomial SVM follows the same procedure illustrated in the RBF kernel SVM calculation. The diagram in Fig. 15 depicts the block diagram of the polynomial kernel SVM where $f(x) = x^n$ is implemented by a series of multipliers, and since this work implemented only the 3rd order polynomial SVM, two serial multipliers are used.

4.1.4. Sigmoid kernel SVM design

The sigmoid kernel SVM follows the same steps as other kernel SVMs, except for the kernel calculation. The polynomial and sigmoid kernels in Fig. 11 share the same operations . The only difference is in the last operation, where the result's hyperbolic tangent (Tanh) is calculated instead of raising the result to the dth degree. The approximation of the hyperbolic tangent function (Tanh) is optimized for efficient hardware performance. Multiple approximations are tested. The 2-term polynomial approximation in [42] provides acceptable accuracy for the tested datasets while occupying low hardware resources and is given

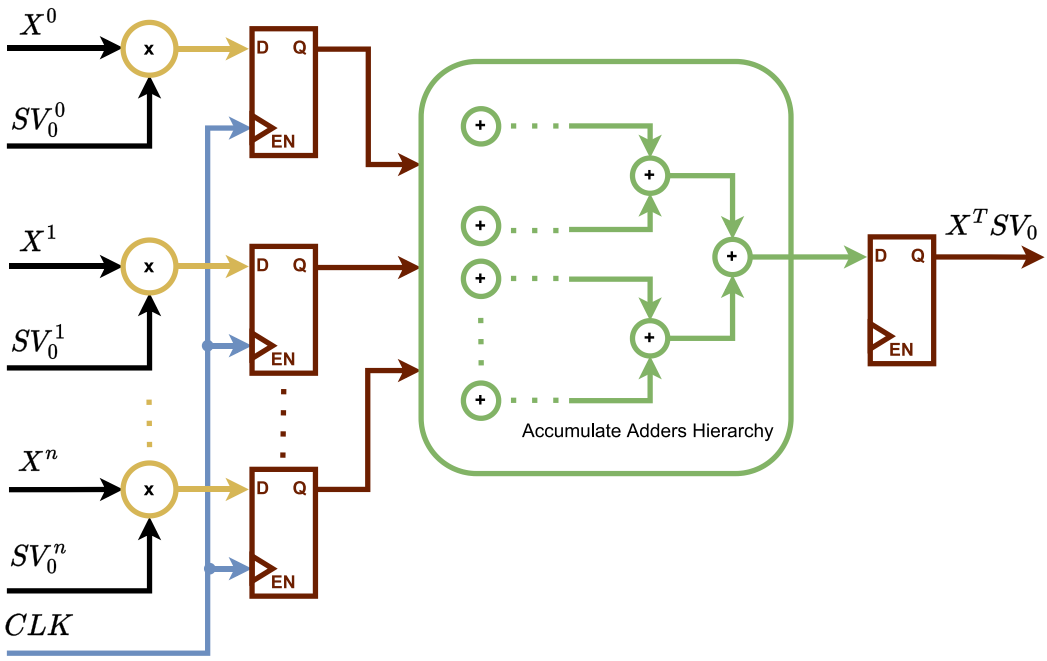


Fig. 9. Dot Product block diagram.

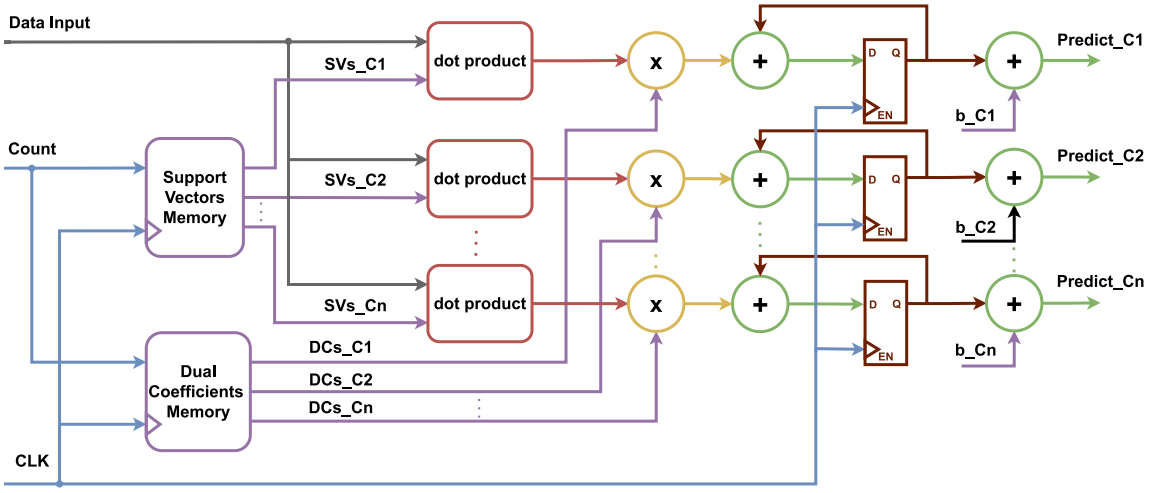


Fig. 10. Linear SVM block diagram.

by:

$$tanh(x) = \begin{cases} x - 0.25sign(x)x^2 & , |x| \leq 2 \\ sign(x) & , \text{otherwise.} \end{cases} \quad (14)$$

This may not be the case for more complex datasets with a high number of features and support vectors, so a more accurate 4-term approximation is developed by fitting a 3rd degree polynomial on the region from −2.5 to 2.5 range of the Tanh curve and is given by:

$$tanh(x) = \begin{cases} 0.06902x^3 - 0.491sign(x)x^2 \\ +1.198x - 0.01859sign(x) & , |x| \leq 2.7765 \\ sign(x) & , \text{otherwise.} \end{cases} \quad (15)$$

This approximation provides better accuracy than the first but with extra hardware cost. Only the positive region is fitted as the negative region is the direct inverse of the positive region, reducing the polynomial degree required to reach the targeted Mean Square Error (MSE). Due to fitting only the positive region, the MSE is lower around zero because of the polynomial function's low fit around the region edges. A third approximation is proposed to solve this problem that combines the advantages of the first two approximations that is given by:

$$tanh(x) = \begin{cases} x - 0.25sign(x)x^2 & , |x| < 0.1 \\ +0.069x^3 - 0.491sign(x)x^2 \\ +1.198x - 0.01859sign(x) & , 0.1 \leq |x| \leq 2.7765 \\ sign(x) & , \text{otherwise.} \end{cases} \quad (16)$$

Figs. 16(a), 16(b), and 16(c) present the first, second, and third approximations plotted for input values from −8 to 8 against exact results obtained from NumPy python library. The results show that approx 2 and 3 have a significantly better relative error than approx 1 while approx 3 has a better relative error around zero than approx 2. Table 5 compares the three approximations regarding accuracy and resources. The first approximation was used for the experiments done on the three datasets, as its accuracy is sufficient for correctly classifying them. If a more accurate solution is required, the most accurate approximation, approx 3, can be used with a relatively significant hardware cost introduced. Fig. 17 presents the hardware block diagram

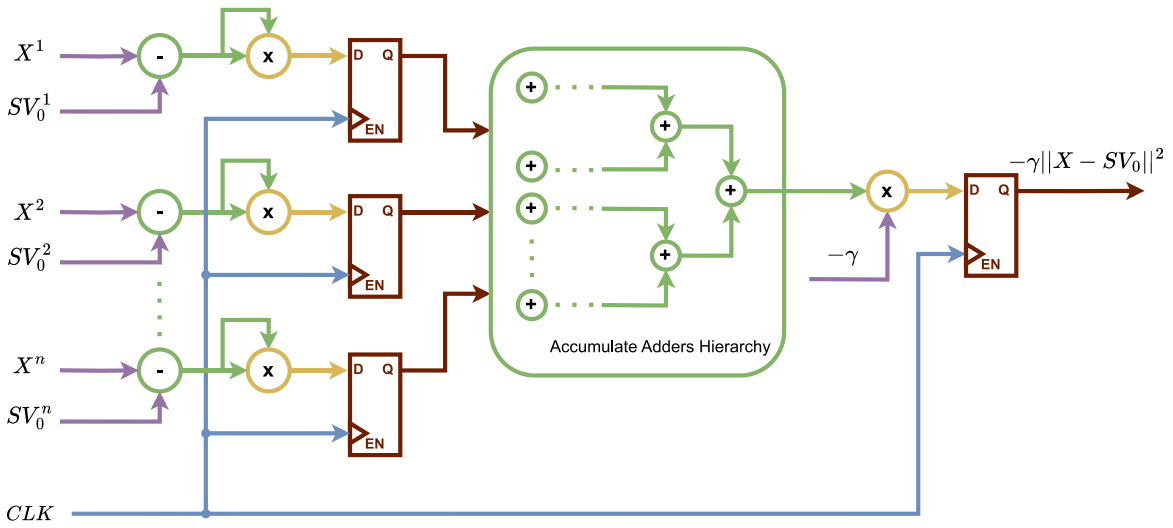


Fig. 11. RBF Kernel's Norm Square component.

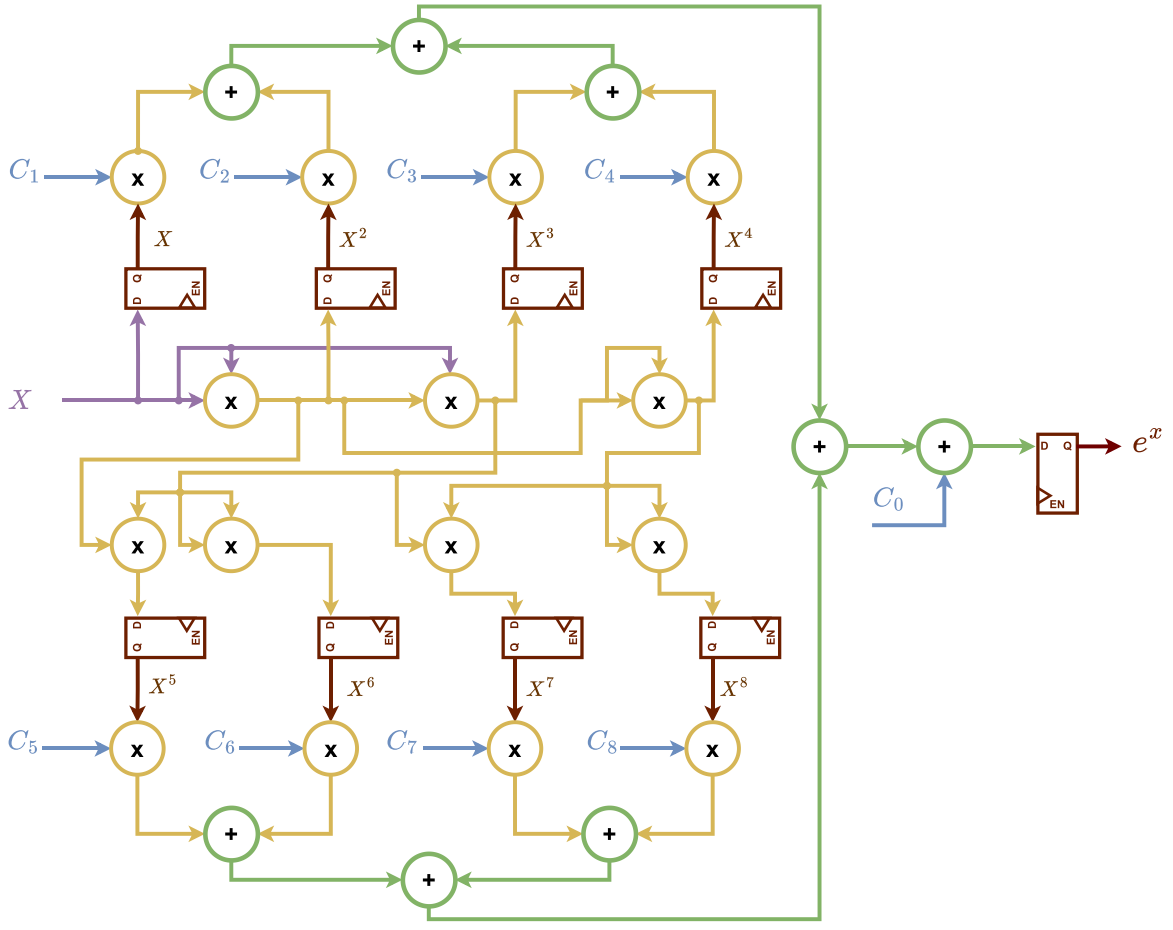


Fig. 12. Proposed exponential function approximation hardware block diagram.

of the first approximation, and Fig. 18 represents the block diagram of the sigmoid kernel SVM path.

4.2. Hardware results and utilization

The classification results of the hardware architecture match the software results for all the tested datasets. The Hardware design here utilizes a separate pipelined path for each classifier. Even though this

Table 5
Tanh approximations comparison.

Metric	Approx 1	Approx 2	Approx 3
MSE	2.06*10⁻4	6.722*10⁻6	5.184*10⁻6
Multipliers	1	5	6
Adders	1	3	4

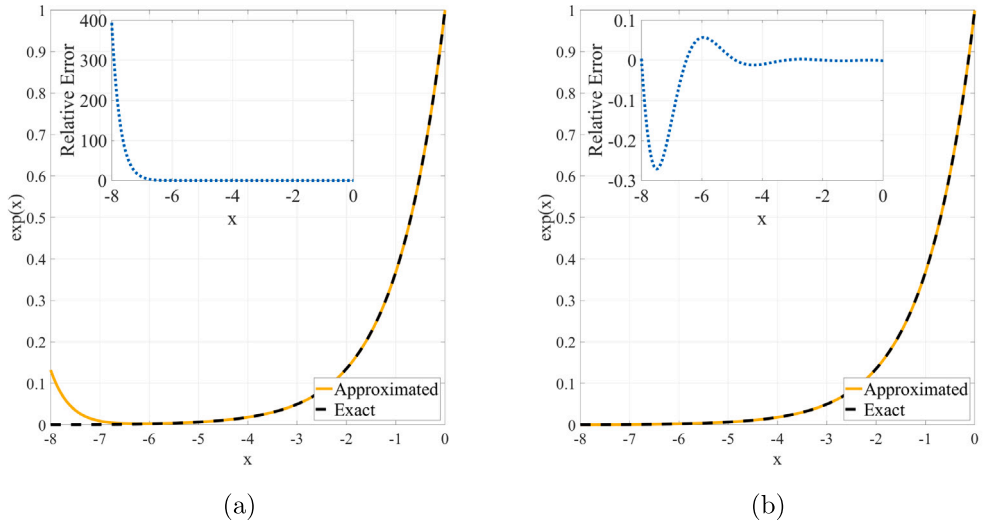


Fig. 13. Exponential function approximations plotted against the exact values (a) 20th degree Taylor, and (b) Proposed.

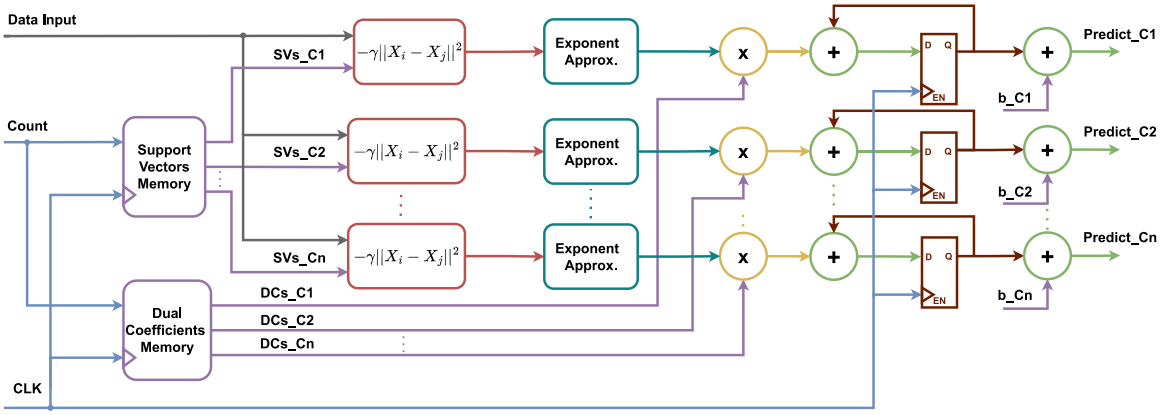


Fig. 14. RBF Kernel SVM Block Diagram.

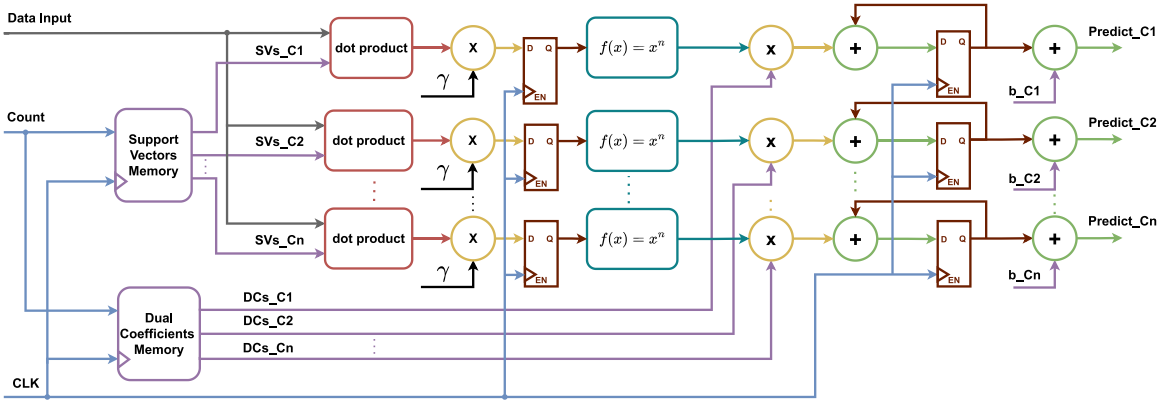


Fig. 15. Polynomial Kernel SVM Block Diagram.

added an extra hardware cost, this cost is justified to save around (C-1/C) of the required clock cycles if only a single pipeline is utilized, which hugely reduces the latency of a complete operation while increasing the throughput of the design by pipelining the path itself. The Area utilization of the hardware depends on the number of features, the number of classes of the dataset, and the kernel used. The RBF kernel is the largest due to the size of the 8th degree exponent approximation. Table 6 presents a performance comparison between multiple SVM models implemented employing the reconfigurable hardware proposed on two different FPGAs and the implementations presented in the literature. Fixed point results of the proposed model are also presented alongside the floating point results. Fixed point implementation can significantly reduce the arithmetic complexity and be used based on the application requirement and its acceptable accuracy range. The results show that the proposed architecture achieves much better power consumption than the other implementations in the literature.

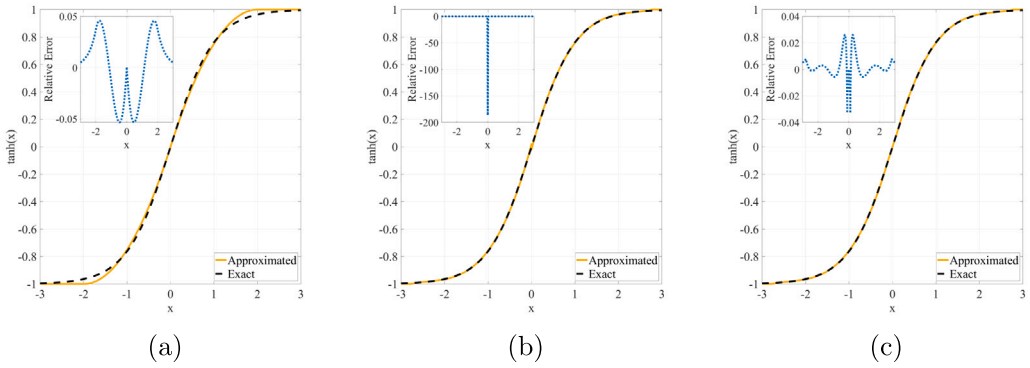


Fig. 16. The three Tanh approximations plotted against the exact values (a) approx 1 [42], (b) approx 2 (Proposed), and (c) approx 3 (Proposed).

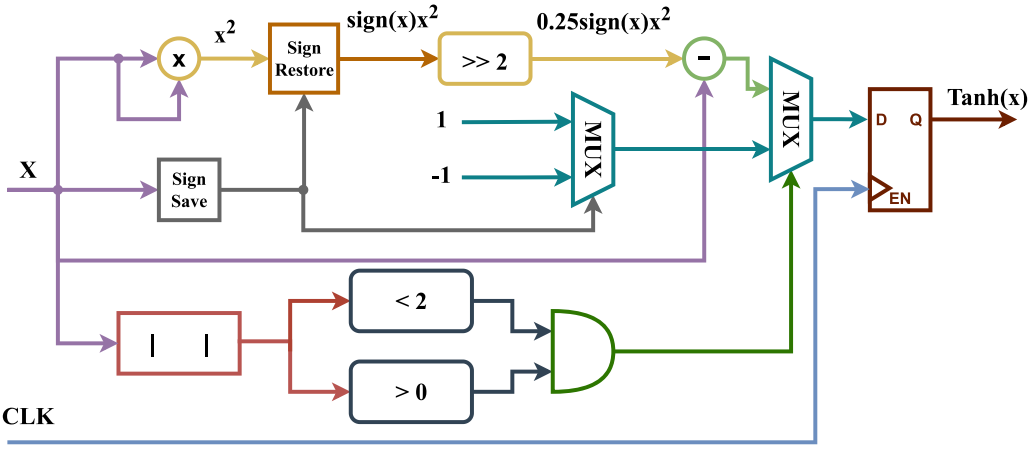


Fig. 17. Tanh approximation one block diagram.

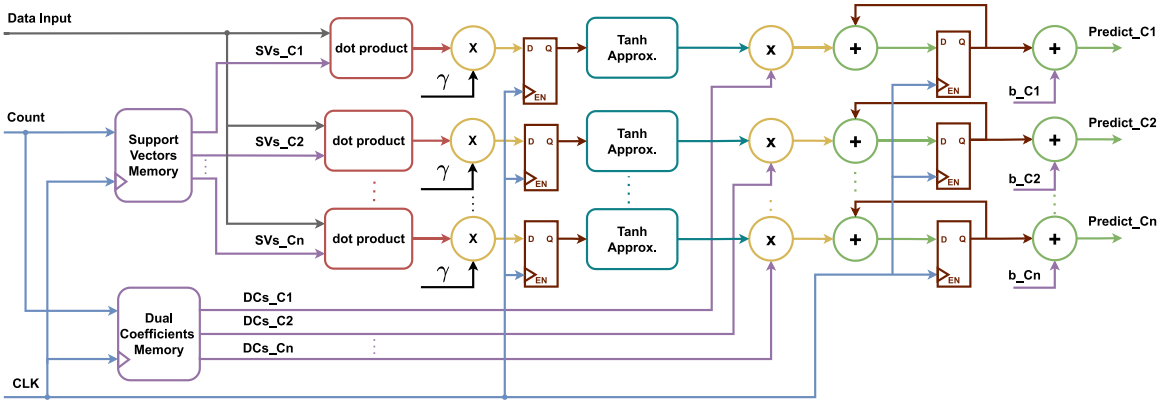


Fig. 18. Sigmoid Kernel SVM Block Diagram.

Furthermore, it efficiently utilizes the FPGA LUT resources even for smaller FPGAs, such as the Artix-7 employed in this work. The largest design utilizes only 7039 out of 63400 LUTs, which is only 11% of the employed Nexys-4's Artix-7 FPGA LUT resources. Moreover, the architecture also achieves high performance, reaching more than 250 MHz frequency. Additionally, due to the design reconfigurability, the design can fit various datasets with minimum parameter changes. This allows it to be easily updated when the application or the dataset used is changed, better leveraging the programmability of the FPGA.

4.3. Hardware experimental results

The hardware architecture in this section is realized and tested on the Nexys 4 board based on Xilinx Artix 7 FPGA. Fig. 19 shows the experimental results for classifying the Iris data set using RBF kernel classifier with OvO multi-class technique. The results were presented on the seven-segment display, where the first three digits from the right represent the prediction of each classifier, and the last digit is the predicted class based on the majority of votes. The experimental

Table 6
Hardware implementation of SVM comparison.

Paper	Arithmetic	LUTs	BRAM	DSPs	Power (W)	FPGA	Freq (MHz)	Classifier
[43]	FP [32 bits]	3414	2	10	1.74	Zynq-7	250	Linear
[44]	FX [40 bits]	748	31	27	–	Virtex-4	202.84	OVO Linear
	FX [38 bits]	9141	99	81	–	Virtex-4	151.286	OVO Gaussian
[45]	FX [32 bits]	122,637	2049	–	15	Virtex-5	200	Binary Gaussian
[46]	FX	35,532	256	59	4.1 - 9.9	Spartan-6	70	Linear
[47]	FX [20 bits]	16,138	576 KB	53	–	Virtex-5	50	OVO RBF
[48]	FX	22,938	160 KB	–	–	Virtex-2 Pro	50	Polynomial
[49]	FX	12,068	140	66	–	Virtex-6	120	Linear
Proposed	FX [32 bits]	1120	0	60	0.296	Kintex-7	158	OVO Linear
	FX [18 bits]	367	0	15	0.238	Kintex-7	250.7	OVO Linear
	FX [32 bits]	5725	0	120	0.323	Kintex-7	63.7	OVO RBF
	FX [18 bits]	981	0	48	0.214	Kintex-7	106.4	OVO RBF
	FP [32 bits]	5402	0	30	0.110	Artix-7	30.5	OVO Linear
	FP [32 bits]	6476	0	48	0.158	Artix-7	31.5	OVO Polynomial
	FP [32 bits]	7039	0	42	0.156	Artix-7	32.14	OVO Sigmoid

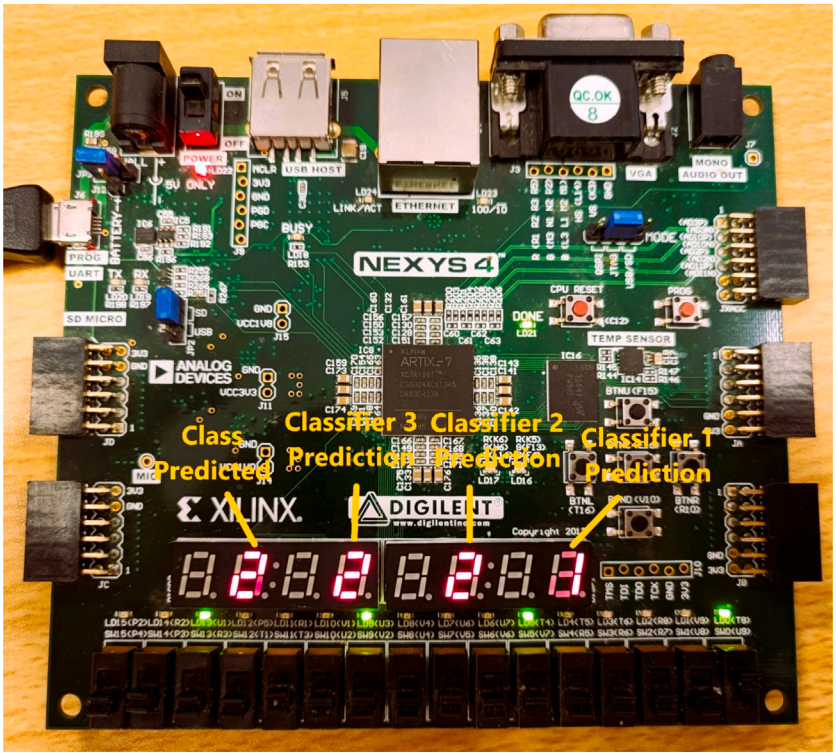


Fig. 19. Hardware Experimental Results for classifying the Iris data set using RBF kernel classifier with OvO generalization technique.

results match the software simulation results without any misclassified samples being detected.

5. Conclusion

This work proposed a reconfigurable hardware implementation of the SVM algorithm for the linear and three kernel cases illustrated in detail and experimentally tested on FPGA. The presented design can be easily adapted to datasets with varying numbers of features and classes, with only the parameters and the memory to be updated to fit any new problem. The results show that the proposed model is efficient in terms of power and area while providing high performance compared to previous work in the literature. The linear SVM architecture is the most efficient method from a hardware implementation perspective. For multi-class problems, OvA reduces the hardware cost for problems with a massive number of classes but performs poorly for imbalanced sets and is better used for balanced datasets with a small number of classes. OvO offers a much better fit for imbalanced datasets and problems with a high number of classes. The hardware experimental results realized on the Artix-7 FPGA typically match the software simulation results for all the tested cases.

CRediT authorship contribution statement

Mohammed H. Yacoub: Conceptualization, Investigation, Methodology, Software, Writing – original draft. **Samar M. Ismail:** Conceptualization, Project administration, Supervision, Writing – review & editing. **Lobna A. Said:** Conceptualization, Project administration, Supervision, Writing – review & editing. **Ahmed H. Madian:** Conceptualization, Supervision. **Ahmed G. Radwan:** Conceptualization, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] I.C.L. Hub, "What is artificial intelligence?" URL https://www.ibm.com/cloud/learn/what-is-artificial-intelligence.

[2] X. Zhang, Y. Qiao, F. Meng, C. Fan, M. Zhang, Identification of maize leaf diseases using improved deep convolutional neural networks, IEEE Access 6 (2018) 30370–30377.

[3] N. Kulkarni, B. Patanwadia, V. Kulkarni, A survey on machine learning techniques for breast cancer diagnosis and detection, in: 2021 3rd International Conference on Advances in Computing, Communication Control and Networking, ICAC3N, 2021, pp. 425–427.

[4] M. Mamdouh, M.A.I. Elrukhsi, A. Khattab, Securing the internet of things and wireless sensor networks via machine learning: A survey, in: 2018 International Conference on Computer and Applications, ICCA, 2018, pp. 215–218.

[5] M.P. Deisenroth, D. Fox, C.E. Rasmussen, Gaussian processes for data-efficient learning in robotics and control, IEEE Trans. Pattern Anal. Mach. Intell. 37 (2) (2015) 408–423.

[6] P.-F. Zhang, Y.-J. Su, C. Wang, Statistical machine learning used in integrated anti-spam system, in: 2007 International Conference on Machine Learning and Cybernetics, 7, 2007, pp. 4055–4058.

[7] S. Safavian, D. Landgrebe, A survey of decision tree classifier methodology, IEEE Trans. Syst. Man Cybern. 21 (3) (1991) 660–674.

[8] J. Zhang, M. Zulkernine, A. Haque, Random-forests-based network intrusion detection systems, IEEE Trans. Syst. Man Cybern. C (Applications and Reviews) 38 (5) (2008) 649–659.

[9] A. Wibawa, A. Kurniawan, D. Murti, R.P. Adiperkasa, S. Putra, S. Kurniawan, Y. Nugraha, Naïve Bayes classifier for journal quartile classification, Int. J. Recent Contributions Eng. Sci. IT (iJES) 7 (2019) 91.

[10] M. Yacoub, S. Ismail, L. Said, H. Madian, A. Radwan, Generic hardware realization of K nearest neighbors on FPGA, 2022, pp. 169–172.

[11] A. Lakshmanarao, M.R. Babu, T.S.R. Kiran, Plant disease prediction and classification using deep learning ConvNets, in: 2021 International Conference on Artificial Intelligence and Machine Vision, AIMV, 2021, pp. 1–6.

[12] H. Drucker, D. Wu, V. Vapnik, Support vector machines for spam categorization, IEEE Trans. Neural Netw. 10 (5) (1999) 1048–1054.

[13] A. Geron, Hands-on machine learning with Scikit-Learn and TensorFlow : concepts, tools, and techniques to build intelligent systems, O'Reilly Media, Sebastopol, CA, 2017.

[14] H. Drucker, C.J.C. Burges, L. Kaufman, A. Smola, V. Vapnik, Support vector regression machines, in: M. Mozer, M. Jordan, T. Petsche (Eds.), Advances in Neural Information Processing Systems, 9, MIT Press, 1996.

[15] A. Ben-Hur, D. Horn, H. Siegelmann, V. Vapnik, Support vector clustering, J. Mach. Learn. Res. 2 (2001) 125–137.

[16] A. Kurani, P. Doshi, A. Vakharia, M. Shah, A comprehensive comparative study of Artificial Neural Network (ANN) and Support Vector Machines (SVM) on stock forecasting, Ann. Data Sci. 10 (1) (2023) 183–208.

[17] A. Roy, S. Chakraborty, Support vector machine in structural reliability analysis: A review, Reliab. Eng. Syst. Saf. 233 (2023) 109126, URL https://www.sciencedirect.com/science/article/pii/S0951832023000418.

[18] H. Li, H. Chen, Y. Li, Q. Chen, X. Fan, S. Li, M. Ma, Prediction of the optical properties in photonic crystal fiber using support vector machine based on radial basis functions, Optik 275 (2023) 170603, URL https://www.sciencedirect.com/science/article/pii/S0030402623000992.

[19] K. Veropoulos, N. Cristianini, C. Campbell, The application of support vector machines to medical decision support: A case study, Adv. Course Artif. Intell. (1999).

[20] A support vector machine-based ensemble algorithm for breast cancer diagnosis, European J. Oper. Res. 267 (2) (2018) 687–699.

[21] J. Verma, M. Nath, P. Tripathi, K. Saini, Analysis and identification of kidney stone using Kth nearest neighbour (KNN) and support vector machine (SVM) classification techniques, Pattern Recognit. Image Anal. 27 (2017) 574–580.

[22] M. Ghanat Bari, C.Y. Ung, C. Zhang, S. Zhu, H. Li, Machine learning-assisted network inference approach to identify a new class of genes that coordinate the functionality of cancer networks, Sci. Rep. 7 (1) (2017) 6993.

[23] N.A. Jebril, H.R. Al-Zoubi, Q.A. Al-Haija, Recognition of handwritten arabic characters using histograms of oriented gradient (HOG), Pattern Recognit. Image Anal. 28 (2018) 321–345.

[24] H.-M. Je, D. Kim, S. Bang, S. Lee, Y. Choi, Human face detection in digital video using SVM ensemble, 2002, pp. 417–421.

[25] J. Julie, J.J. Athanesious, T. Santhosh, B. Vigneshwar, Novel weed detection algorithm for sesame crop using region-based CNN with support vector machine, in: 2021 4th International Conference on Computing and Communications Technologies, ICCCT, 2021, pp. 247–251.

[26] S.M. Mohamed, W.S. Sayed, L.A. Said, A.G. Radwan, Reconfigurable FPGA realization of fractional-order chaotic systems, IEEE Access 9 (2021) 89376–89389.

[27] W.S. Sayed, M. Roshdy, L.A. Said, A.G. Radwan, Design and FPGA verification of custom-shaped chaotic attractors using rotation, offset boosting and amplitude control, IEEE Trans. Circuits Syst. II 68 (11) (2021) 3466–3470.

[28] A.M. Abdelaty, M. Roshdy, L.A. Said, A.G. Radwan, Numerical simulations and FPGA implementations of fractional-order systems based on product integration rules, IEEE Access 8 (2020) 102093–102105.

[29] M.F. Tolba, L.A. Said, A.H. Madian, A.G. Radwan, FPGA implementation of fractional-order integrator and differentiator based on Grünwald Letnikov's definition, in: 2017 29th International Conference on Microelectronics, ICM, 2017, pp. 1–4.

[30] N. Sulaiman, Z. Obaid, M.H. Marhaban, M.N. Hamidon, Design and implementation of FPGA-based systems -A review, Aust. J. Basic Appl. Sci. 3 (2009).

[31] C. Maxfield, FPGAs: Instant access, ISBN: 9780750689748, 2008, pp. 61–73.

[32] J. Platt, Fast training of support vector machines using sequential minimal optimization, in: Advances in Kernel Methods - Support Vector Learning, MIT Press, 1998.

[33] T. Hofmann, B. Sch, A. Smola, A review of kernel methods in machine learning, 2006.

[34] N. Cristianini, J. Shawe-Taylor, An introduction to support vector machines and other kernel-based learning methods, Cambridge University Press, 2000.

[35] A. Janosi, W. Steinbrunn, M. Pfisterer, R. Detrano, UCI Heart Disease Data Set.

[36] R. Fisher, UCI Iris Data Set.

[37] A. Sharma, Mobile Prices Data Set.

[38] IEEE standard for floating-point arithmetic, IEEE Std 754-2019 (Revision of IEEE 754-2008) (2019) 1–84.

[39] S.M. Ismail, A.M. Ghidan, P.W. Zaki, Novel chaotic random memory indexing steganography on FPGA, AEU - Int. J. Electron. Commun. 125 (2020) 153367, URL https://www.sciencedirect.com/science/article/pii/S1434841120305586.

[40] O.H. Moustafa, S.M. Ismail, FPGA-based floating point fractional order image edge detection, in: 2019 15th International Computer Engineering Conference, ICENCO, 2019, pp. 91–94.

[41] H.S. Hassan, S.M. Ismail, CLA based floating-point adder suitable for chaotic generators on FPGA, in: 2018 30th International Conference on Microelectronics, ICM, 2018, pp. 299–302.

[42] C.-W. Lin, J.-S. Wang, A digital circuit design of hyperbolic tangent sigmoid function for neural networks, in: 2008 IEEE International Symposium on Circuits and Systems, ISCAS, 2008, pp. 856–859.

[43] S. Afifi, H. GholamHosseini, R. Sinha, SVM classifier on chip for melanoma detection, in: 2017 39th Annual International Conference of the IEEE Engineering in Medicine and Biology Society, EMBC, 2017, pp. 270–274.
[44] D. Mahmoodi, A. Soleimani, H. Khosravi, M. Taghizadeh, FPGA simulation of linear and nonlinear support vector machine, JSEA 4 (2011) 320–328.
[45] M. Pietron, M. Wielgosz, D. Zurek, E. Jamro, K. Wiatr, Comparison of GPU and FPGA implementation of SVM algorithm for fast image segmentation, in: H. Kubátová, C. Hochberger, M. Daněk, B. Sick (Eds.), Architecture of Computing Systems – ARCS 2013, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, pp. 292–302.
[46] C. Kyrkou, C.-S. Bouganis, T. Theocharides, M.M. Polycarpou, Embedded hardware-efficient real-time classification with cascade support vector machines, IEEE Trans. Neural Netw. Learn. Syst. 27 (1) (2016) 99–112.
[47] M. Qasaimeh, A. Sagahyroon, T. Shanableh, FPGA-based parallel hardware architecture for real-time image classification, IEEE Trans. Comput. Imaging 1 (1) (2015) 56–70.
[48] R.-R. Roberto, D. Houzet, D. Dragomirescu, F. Carlier, O. Salim, Object recognition system-on-chip using the support vector machines, EURASIP J. Adv. Signal Process. 2005 (2005).
[49] T. Kryjak, M. Komorkiewicz, M. Gorgon, FPGA implementation of real-time head-shoulder detection using local binary patterns, SVM and foreground object detection, in: Conference on Design and Architectures for Signal and Image Processing, DASIP, 2012, pp. 1–8.